

La Libreria `mimetypes` di Python - Guida Pratica

Indice

1. [Introduzione](#)
 2. [Concetti Fondamentali](#)
 3. [Funzioni Principali](#)
 4. [Esempi Pratici](#)
 5. [Casi d'Uso Reali](#)
 6. [Esercizi](#)
-

Introduzione

La libreria `mimetypes` è parte della **standard library di Python** e serve a **mappare le estensioni dei file ai loro tipi MIME (Multipurpose Internet Mail Extensions)**.

Un tipo MIME è uno standard che identifica il tipo di contenuto di un file. Esempi:

- `image/png` → immagini PNG
- `application/pdf` → documenti PDF
- `text/plain` → file di testo
- `video/mp4` → video MP4

Perché è utile?

- Configurare intestazioni HTTP corrette (`Content-Type`)
- Validare file caricati dagli utenti
- Determinare come aprire un file automaticamente
- Gestire download web

Installazione: Non è necessaria! È già inclusa in Python.

```
import mimetypes
```

Concetti Fondamentali

Che cos'è un tipo MIME?

Un tipo MIME ha la struttura: `tipo/sottotipo`

Tipo	Sottotipo	Completo	Descrizione
text	plain	<code>text/plain</code>	File di testo semplice
text	html	<code>text/html</code>	Pagina HTML
image	jpeg	<code>image/jpeg</code>	Immagine JPEG
image	png	<code>image/png</code>	Immagine PNG
application	pdf	<code>application/pdf</code>	Documento PDF
application	json	<code>application/json</code>	Dati JSON
video	mp4	<code>video/mp4</code>	Video MP4
audio	mpeg	<code>audio/mpeg</code>	Audio MP3

Che cosa fa `mimetypes` ?

Legge l'**estensione del file** e la confronta con un database interno per restituire il tipo MIME.

Importante: Non analizza il contenuto del file, solo l'estensione!

Funzioni Principali

1. `guess_type(filename)`

La funzione principale che indovina il tipo MIME.

Sintassi:

```
mimetypes.guess_type(filename, strict=True)
```

Ritorna:

- Una tupla `(tipo_mime, encoding)`
- Esempio: `('application/pdf', None)`

Parametri:

- `filename` : percorso o nome del file
- `strict` : se `True` usa solo tipi MIME standard (default)

Esempi:

```
import mimetypes

# File comuni
mimetypes.guess_type('documento.pdf')
# Ritorna: ('application/pdf', None)

mimetypes.guess_type('foto.jpg')
# Ritorna: ('image/jpeg', None)

mimetypes.guess_type('archivio.tar.gz')
# Ritorna: ('application/x-tar', 'gzip') # gz è l'encoding!

mimetypes.guess_type('sconosciuto.xyz')
# Ritorna: (None, None) # Tipo non riconosciuto
```

2. `guess_extension(type, strict=True)`

Fa l'operazione inversa: da tipo MIME a estensione.

```
mimetypes.guess_extension('image/jpeg')
# Ritorna: '.jpeg'

mimetypes.guess_extension('application/pdf')
# Ritorna: '.pdf'

mimetypes.guess_extension('text/plain')
# Ritorna: '.txt'
```

3. `add_type(type, ext, strict=True)`

Registra una mappatura personalizzata.

```
# Aggiungere un tipo MIME personalizzato
mimetypes.add_type('application/x-custom', '.custom')

# Ora funziona:
```

```
mimetypes.guess_type('file.custom')  
# Ritorna: ('application/x-custom', None)
```

4. `init()`

Forza l'inizializzazione del database MIME (di solito automatica).

```
mimetypes.init()
```

5. `inited`

Variabile che indica se il database è inizializzato.

```
if mimetypes.inited:  
    print("Database MIME già caricato")  
else:  
    mimetypes.init()
```

Esempi Pratici

Esempio 1: Riconoscere Diversi Tipi di File

```
import mimetypes  
  
file_list = [  
    'documento.pdf',  
    'immagine.png',  
    'video.mp4',  
    'musica.mp3',  
    'pagina.html',  
    'dati.json',  
    'foglio.xlsx',  
    'archivio.zip'  
]  
  
for filename in file_list:  
    mime_type, encoding = mimetypes.guess_type(filename)  
    print(f"{filename:20} → {mime_type}")
```

Output:

documento.pdf	→ application/pdf
immagine.png	→ image/png
video.mp4	→ video/mp4
musica.mp3	→ audio/mpeg
pagina.html	→ text/html
dati.json	→ application/json
foglio.xlsx	→ application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
archivio.zip	→ application/zip

Esempio 2: Funzione di Validazione File

```
import mimetypes

def valida_file_upload(filename, tipi_consentiti):
    """
    Valida se un file ha un tipo MIME consentito.

    Args:
        filename: nome del file
        tipi_consentiti: lista di tipi MIME accettati

    Returns:
        bool: True se il file è valido, False altrimenti
    """
    mime_type, _ = mimetypes.guess_type(filename)

    if mime_type is None:
        print(f"✗ Tipo sconosciuto: {filename}")
        return False

    if mime_type in tipi_consentiti:
        print(f"✓ {filename} ({mime_type}) - ACCETTATO")
        return True
    else:
        print(f"✗ {filename} ({mime_type}) - RIFIUTATO")
        return False

# Utilizzo: Solo immagini
tipi_immagini = ['image/jpeg', 'image/png', 'image/gif']
valida_file_upload('foto.jpg', tipi_immagini)      # ✓
valida_file_upload('documento.pdf', tipi_immagini) # ✗

# Utilizzo: Solo documenti
tipi_documenti = ['application/pdf', 'text/plain', 'application/msword']
```

```
valida_file_upload('relazione.pdf', tipi_documenti) #   
valida_file_upload('musica.mp3', tipi_documenti) # 
```

Esempio 3: Organizzare File per Categoria

```
import mimetypes  
from pathlib import Path  
  
def categorizza_file(file_list):  
    """  
    Organizza i file per categoria in base al tipo MIME.  
    """  
    categorie = {  
        'immagini': [],  
        'documenti': [],  
        'audio': [],  
        'video': [],  
        'archivi': [],  
        'altro': []  
    }  
  
    for filename in file_list:  
        mime_type, _ = mimetypes.guess_type(filename)  
  
        if mime_type is None:  
            categorie['altro'].append(filename)  
        elif mime_type.startswith('image/'):   
            categorie['immagini'].append(filename)  
        elif mime_type.startswith('audio/'):   
            categorie['audio'].append(filename)  
        elif mime_type.startswith('video/'):   
            categorie['video'].append(filename)  
        elif 'pdf' in mime_type or 'word' in mime_type or 'sheet' in mime_type:  
            categorie['documenti'].append(filename)  
        elif 'zip' in mime_type or 'tar' in mime_type or 'rar' in mime_type:  
            categorie['archivi'].append(filename)  
        else:  
            categorie['altro'].append(filename)  
  
    # Stampa i risultati  
    for categoria, file_list in categorie.items():  
        if file_list:  
            print(f"\n{categoria.upper()}:")  
            for f in file_list:  
                print(f" - {f}")
```

```
# Test
file_test = [
    'foto.jpg',
    'documento.pdf',
    'canzone.mp3',
    'film.mp4',
    'backup.zip',
    'script.py',
    'readme.txt'
]

categorizza_file(file_test)
```

Output:

```
IMMAGINI:
- foto.jpg

DOCUMENTI:
- documento.pdf

AUDIO:
- canzone.mp3

VIDEO:
- film.mp4

ARCHIVI:
- backup.zip

ALTRO:
- script.py
- readme.txt
```

Esempio 4: Simulare un Server Web

```
import mimetypes

def simula_header_http(filename):
    """
    Simula l'header HTTP che un server web invierebbe.
    """
    mime_type, _ = mimetypes.guess_type(filename)
```

```

if mime_type is None:
    mime_type = 'application/octet-stream' # Tipo generico

print(f"GET /{filename} HTTP/1.1")
print(f"Content-Type: {mime_type}")
print(f"Content-Length: [dimensione file]")
print()

# Esempi
files = ['index.html', 'style.css', 'script.js', 'immagine.png', 'documento.pdf']

for f in files:
    simula_header_http(f)

```

Output:

```

GET /index.html HTTP/1.1
Content-Type: text/html
Content-Length: [dimensione file]

GET /style.css HTTP/1.1
Content-Type: text/css
Content-Length: [dimensione file]

GET /script.js HTTP/1.1
Content-Type: application/javascript
Content-Length: [dimensione file]

GET /immagine.png HTTP/1.1
Content-Type: image/png
Content-Length: [dimensione file]

GET /documento.pdf HTTP/1.1
Content-Type: application/pdf
Content-Length: [dimensione file]

```

Esempio 5: Lavorare con Percorsi di File

```

import mimetypes
from pathlib import Path

def analizza_cartella(percorso_cartella):
    """
    Analizza tutti i file in una cartella.
    """

```

```

"""
cartella = Path(percorso_cartella)

for file in cartella.glob('*'):
    if file.is_file():
        mime_type, encoding = mimetypes.guess_type(file)

        info = f"Nome: {file.name}"
        info += f"\n  Tipo MIME: {mime_type or 'Sconosciuto'}"

        if encoding:
            info += f"\n  Encoding: {encoding}"

        info += f"\n  Dimensione: {file.stat().st_size} bytes"
        print(info)
        print()

# Utilizzo (crea una cartella di test prima!)
# analizza_cartella('./miei_file')

```

Esempio 6: Tipi MIME Personalizzati

```

import mimetypes

# Registrare tipi MIME personalizzati
mimetypes.add_type('application/x-python-source', '.py')
mimetypes.add_type('application/x-yaml', '.yaml')
mimetypes.add_type('application/x-yaml', '.yaml')
mimetypes.add_type('application/x-custom-app', '.myapp')

# Ora questi funzionano:
print(mimetypes.guess_type('script.py'))      # ('application/x-python-source', None)
print(mimetypes.guess_type('config.yaml'))     # ('application/x-yaml', None)
print(mimetypes.guess_type('app.myapp'))       # ('application/x-custom-app', None)

```

Casi d'Uso Reali

Caso 1: API REST per Upload File

```

import mimetypes
from typing import List

```

```

class GestoreUpload:
    def __init__(self, tipi_consentiti: List[str] = None):
        self.tipi_consentiti = tipi_consentiti or [
            'image/jpeg',
            'image/png',
            'application/pdf'
        ]

    def valida(self, filename: str) -> dict:
        """Valida un file prima dell'upload."""
        mime_type, _ = mimetypes.guess_type(filename)

        return {
            'filename': filename,
            'mime_type': mime_type,
            'valido': mime_type in self.tipi_consentiti,
            'messaggio': (
                '✅ File accettato'
                if mime_type in self.tipi_consentiti
                else '❌ Tipo di file non consentito'
            )
        }

# Utilizzo
gestore = GestoreUpload()
print(gestore.valida('documento.pdf')) # {'filename': 'documento.pdf', 'mime_type': 'application/pdf', 'valido': True, 'messaggio': '✅ File accettato'}
print(gestore.valida('script.exe'))    # {'filename': 'script.exe', 'mime_type': None, 'valido': False, 'messaggio': '❌ Tipo di file non consentito'}

```

Caso 2: Comando CLI per Analizzare File

```

import mimetypes
import sys

def main():
    if len(sys.argv) < 2:
        print("Uso: python script.py <file>")
        sys.exit(1)

    filename = sys.argv[1]
    mime_type, encoding = mimetypes.guess_type(filename)

    print(f"File: {filename}")
    print(f"Tipo MIME: {mime_type or 'Sconosciuto'}")

```

```

if encoding:
    print(f"Encoding: {encoding}")

# Suggerimento di estensione
if mime_type:
    estensione = mimetypes.guess_extension(mime_type)
    print(f"Estensione suggerita: {estensione}")

if __name__ == '__main__':
    main()

```

Esercizi

Esercizio 1: Filtrare File per Tipo

Crea una funzione che accetta una lista di file e ritorna solo quelli di un certo tipo.

```

def filtra_per_tipo(file_list, tipo_ricercato):
    """
    Filtra file per tipo MIME.

    TODO: Implementare
    """
    pass

# Test
file_list = ['foto.jpg', 'documento.pdf', 'foto2.png', 'relazione.pdf']
print(filtra_per_tipo(file_list, 'image/jpeg')) # ['foto.jpg']

```

Soluzione:

```

def filtra_per_tipo(file_list, tipo_ricercato):
    risultati = []
    for filename in file_list:
        mime_type, _ = mimetypes.guess_type(filename)
        if mime_type == tipo_ricercato:
            risultati.append(filename)
    return risultati

```

Esercizio 2: Contare File per Categoria

Crea una funzione che conta quanti file appartengono a cada categoria MIME.

```
def conta_per_categoria(file_list):
    """
    Conta il numero di file per categoria.

    TODO: Implementare
    """
    pass

# Test
file_list = ['foto.jpg', 'video.mp4', 'foto2.png', 'film.mp4', 'musica.mp3']
print(conta_per_categoria(file_list))
# Output: {'image': 2, 'video': 2, 'audio': 1}
```

Soluzione:

```
def conta_per_categoria(file_list):
    contatori = {}
    for filename in file_list:
        mime_type, _ = mimetypes.guess_type(filename)
        if mime_type:
            categoria = mime_type.split('/')[0] # 'image', 'video', 'audio'...
            contatori[categoria] = contatori.get(categoria, 0) + 1
    return contatori
```

Esercizio 3: Verificare Integrità Estensione

Crea una funzione che verifica se l'estensione di un file corrisponde al suo tipo MIME.

```
def verifica_estensione(filename, mime_type):
    """
    Verifica se l'estensione del file è coerente con il tipo MIME fornito.

    TODO: Implementare
    """
    pass

# Test
print(verifica_estensione('foto.jpg', 'image/jpeg')) # True
print(verifica_estensione('foto.png', 'image/jpeg')) # False
print(verifica_estensione('documento.pdf', 'application/pdf')) # True
```

Soluzione:

```
def verifica_estensione(filename, mime_type):
    estensione_suggerita = mimetypes.guess_extension(mime_type)
    estensione_file = '.' + filename.split('.')[-1] if '.' in filename else None
    return estensione_file == estensione_suggerita
```

Esercizio 4: Rinominare File con Estensione Corretta

Crea una funzione che rinomina i file aggiungendo l'estensione corretta in base al tipo MIME.

```
def rinomina_con_estensione_giusta(filename, mime_type):
    """
    Rinomina un file aggiungendo l'estensione corretta.

    TODO: Implementare
    """
    pass

# Test
print(rinomina_con_estensione_giusta('documento', 'application/pdf'))
# Output: 'documento.pdf'

print(rinomina_con_estensione_giusta('immagine.txt', 'image/png'))
# Output: 'immagine.png'
```

Soluzione:

```
def rinomina_con_estensione_giusta(filename, mime_type):
    # Estrai il nome senza estensione
    nome_senza_ext = filename.split('.')[0]

    # Ottieni l'estensione corretta
    estensione = mimetypes.guess_extension(mime_type)

    # Ritorna il nuovo nome
    return nome_senza_ext + estensione if estensione else filename
```

Pitfalls e Avvertenze

Affidabilità Limitata

`mimetypes` si basa SOLO sull'estensione, non sul contenuto!

```
# Un file rinominato in .pdf sarà riconosciuto come PDF anche se non lo è
mimetypes.guess_type('immagine_falsa.pdf') # ('application/pdf', None)
# Ma potrebbe essere un'immagine!
```

Soluzione: Per validazioni critiche, usare librerie come `python-magic` :

```
pip install python-magic
```

File Senza Estensione

File senza estensione ritornano `None` :

```
mimetypes.guess_type('Makefile') # (None, None)
```

Soluzione: Gestire il caso `None` :

```
mime_type, _ = mimetypes.guess_type(filename)
mime_type = mime_type or 'application/octet-stream'
```

Encoding Compresso

I file compressi hanno sia tipo MIME che encoding:

```
mimetypes.guess_type('archivio.tar.gz')
# ('application/x-tar', 'gzip')
```

Non dimenticare di gestire l'encoding!

Risorse Utili

- **Documentazione ufficiale:** <https://docs.python.org/3/library/mimetypes.html>
- **Lista MIME types:** <https://www.iana.org/assignments/media-types/media-types.xhtml>

- **python-magic:** Alternativa più affidabile per riconoscere tipi di file
 - **file command:** Strumento Linux per identificare tipi di file dal contenuto
-

Conclusione

La libreria `mimetypes` è semplice ma potente per lavori leggeri. Ricorda:

✅ Usa per:

- Configurare header HTTP
- Categorizzare file velocemente
- Validazioni non critiche di sicurezza

❌ Non usare per:

- Validazioni di sicurezza critiche
- Identificazione affidabile di file modificati
- Determinare azioni su file da utenti non fidati

Buona pratica! 🚀